Laboratorio 1 - 2026

Estructura de Datos y Complejidad Algorítmica

Introducción

Las listas enlazadas son una estructura de datos importante en la programación, y en C se pueden implementar fácilmente utilizando punteros. En una lista enlazada, cada nodo contiene un valor y un puntero que apunta al siguiente nodo en la lista. El último nodo en la lista apunta a `NULL`, lo que indica que la lista ha llegado a su fin.

Para crear una lista enlazada en C, primero se define una estructura para el nodo que contiene el valor y el puntero al siguiente nodo:

Listing 1: Estructura de un nodo

```
struct Nodo {
    int valor;
    struct Nodo* siguiente;
};
```

A continuación, se puede crear una función para agregar un nuevo nodo a la lista:

Listing 2: Función para agregar un nodo

```
void agregar(struct Nodo** cabeza, int valor) {
    struct Nodo* nuevoNodo = malloc(sizeof(struct Nodo));
    nuevoNodo->valor = valor;
    nuevoNodo->siguiente = *cabeza;
    *cabeza = nuevoNodo;
}
```

Esta función crea un nuevo nodo, establece su valor y lo hace apuntar al nodo que actualmente es la cabeza de la lista. Luego, actualiza el puntero de la cabeza de la lista para apuntar al nuevo nodo.

También se puede crear una función para imprimir los valores de todos los nodos en la lista:

Listing 3: Función para imprimir una lista

```
void imprimir(struct Nodo* cabeza) {
    struct Nodo* actual = cabeza;
    while (actual != NULL) {
        printf("%d-", actual->valor);
```

```
        actual = actual->siguiente;
    }
}
```

Esta función comienza en la cabeza de la lista y recorre cada nodo en la lista, imprimiendo su valor y luego avanzando al siguiente nodo. La función continúa hasta que alcanza el final de la lista (cuando el puntero del último nodo apunta a `NULL`).

Las listas enlazadas en C pueden ser una herramienta muy útil en la programación para almacenar y manipular datos de manera eficiente. Con las estructuras y funciones adecuadas, se pueden crear listas enlazadas para una amplia variedad de aplicaciones.

Objetivo

Implementar y analizar estructuras de datos fundamentales en C: listas enlazadas, pilas y árboles binarios, utilizando punteros y estructuras `struct`. Además, vincular estas implementaciones con problemas computacionales y su eficiencia algorítmica.

Parte 1: Listas Enlazadas

- Implementar una lista enlazada en C.
- Incluir las funciones:
  - `agregar()`
  - `imprimir()`
  - `recorrer()`
  - `eliminar()`
- **Pregunta extra:** `**` en la función `agregar()`: ¿Qué significa?

Parte 2: Pilas con Listas Enlazadas

- Investigar la implementación de pilas utilizando listas enlazadas.
- Programar las operaciones:
  - `crear()`
  - `agregar()` (push)
  - `borrar()` (pop)
- Comentar un problema computacional que se resuelva con pilas y justificar su orden algorítmico.

Parte 3: Colas con Listas Enlazadas

- Investigar la implementación de colas utilizando listas enlazadas.
- Programar las operaciones:
  - `crear()`
  - `encolar()` (enqueue)
  - `desencolar()` (dequeue)
- Implementar una aplicación concreta que use colas (por ejemplo, sistema de atención de clientes).
- Comentar un problema computacional que se resuelva con colas y justificar su orden algorítmico.

Parte 4: Árboles Binarios

- Investigar la implementación de árboles binarios utilizando listas enlazadas.
- Comentar un problema computacional que se resuelva con árboles binarios y justificar su orden algorítmico.

Entrega

- **Fecha:** Martes 14 de Abril.
- Entregar un archivo por cada elemento (1 pdf, 1 ppt, 1 archivo .c)
  - Informe (Word o PDF)
  - Presentación (PowerPoint)
  - Archivos de código fuente en C
- **Máximo 3 personas, mínimo 1 persona**

Evaluación

Ponderaciones: 40% Laboratorio + 60% Presentación oral.

Rúbrica de Evaluación del Laboratorio

Criterio	Excelente (3)	Aceptable (2)	Insuficiente (1)	No logrado (0)
Implementación de Listas Enlazadas	Implementa correctamente todas las funciones requeridas (agregar, imprimir, recorrer, eliminar).	Implementa todas las funciones con errores menores.	Implementa solo una o dos funciones, con errores.	No implementa ninguna función.
Pilas con Listas Enlazadas	Código funcional completo, investigación documentada y ejemplo de aplicación con justificación del orden.	Faltan uno de los elementos o hay errores relevantes en alguno.	Solo entrega uno de los elementos solicitados o entrega incompleta.	No entrega el trabajo.
Colas con Listas Enlazadas	Código funcional completo, investigación documentada y ejemplo de aplicación con justificación del orden.	Faltan uno de los elementos o hay errores relevantes en alguno.	Solo entrega uno de los elementos solicitados o entrega incompleta.	No entrega el trabajo.
Árboles Binarios	Investigación completa con comentario claro sobre un problema computacional y su orden.	Faltan aspectos del análisis o el comentario es poco claro.	Comenta con errores o superficialmente.	No define nada de lo pedido.

Rúbrica Presentación Oral

Criterio	Deficiente (0–59)	Suficiente (60–79)	Sobresaliente (80–100)
Contenido y estructura	Incoherente	Con algunas lagunas	Clara y organizada
Claridad de expresión	Poco clara	Adecuada	Clara y efectiva
Uso de visuales	Irrelevantes	Pocos y poco relevantes	Relevantes y complementarios

Escala: 60% equivale a nota 4.0 (5 puntos en la presentación).